Timéo Gogolachvili
Rémi Mathivanan

Projet Deep-Learning et IA générative :

Scene-Aware Multimodal Pedestrian Trajectory Prediction

Timéo Gogolachvili
Rémi Mathivanan

Lien Github

https://github.com/Tonpireami/Project-Trajectory-Prediction

Introduction

Dans ce projet de Deep-Learning et d'IA générative, notre objectif est d'utiliser plusieurs modèles d'IA pour prédire les trajectoires futures possibles de piétons dans des scènes urbaines filmées en vue aérienne en se basant sur leurs 8 dernières positions.

Cette tâche peut s'avérer compliquée, car lesdits piétons peuvent avoir des trajectoires imprévisibles, comme tourner subitement ou s'arrêter, et dépendent de leur environnement : ils auront tendance à s'éviter entre eux et seront bloqués par les obstacles sur le chemin.

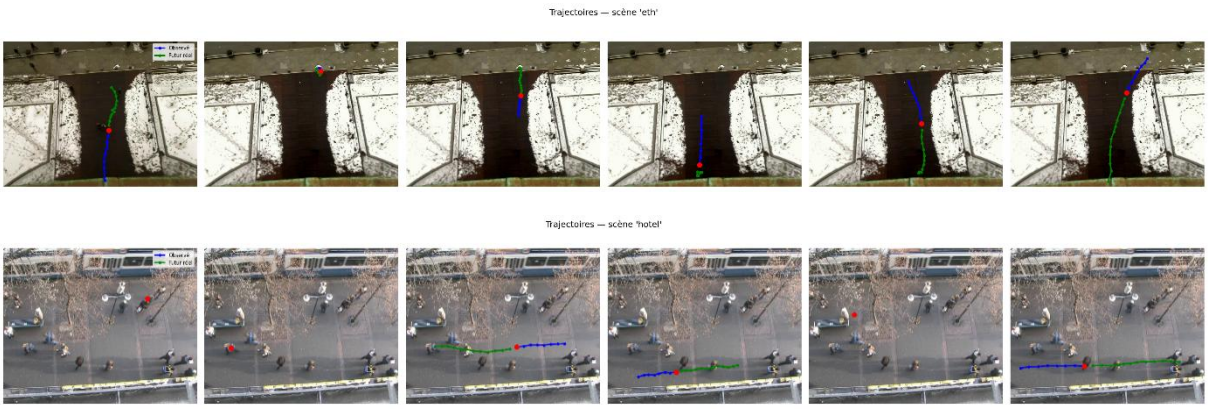
Nous utiliserons donc trois modèles différents pour essayer de prédire les trajectoires. La baseline est le modèle le plus simple, qui prédit la trajectoire comme si le piéton continuait à marcher tout droit. Le LSTM apprend les patterns de mouvement et propose une continuation en conséquence. Enfin, le GAN propose plusieurs trajectoires possibles pour un même piéton.

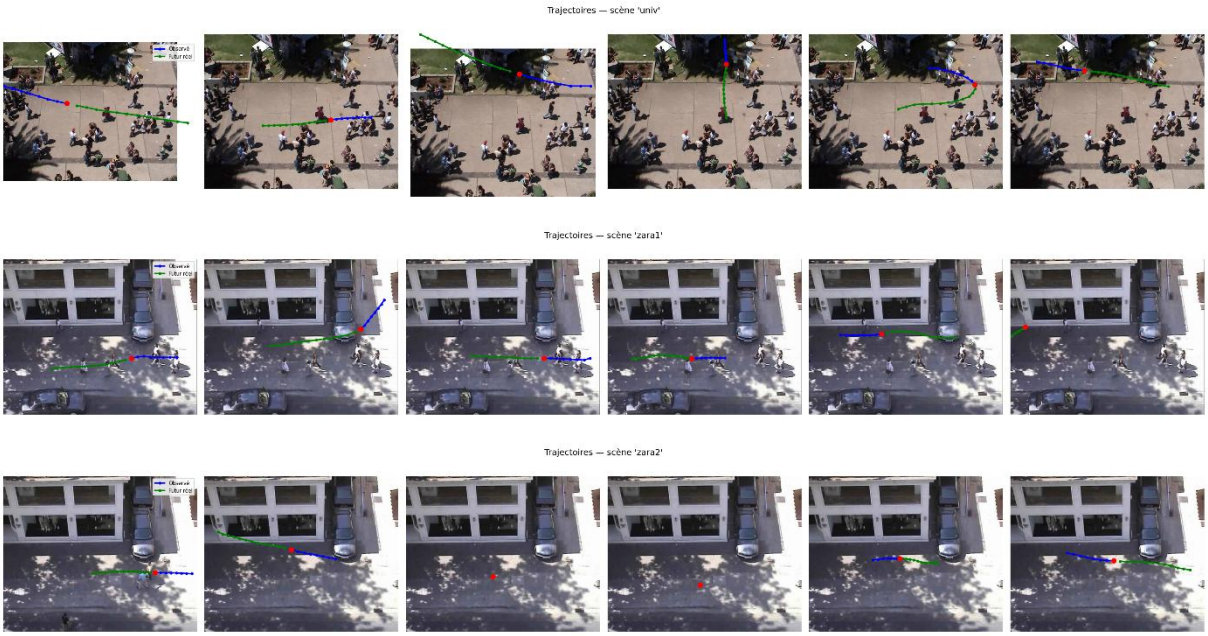
Dataset & données

Le dataset ETH/UCY, que nous utilisons, regroupe les trajectoires de piétons extraites de 5 vidéos dans 5 environnements urbains différents : ETH, Hotel, Université, Zara1 et Zara2.

Ces données nous fournissent les coordonnées réelles en mètres de chaque piéton, découpées en séquences de 20 pas : 8 positions observées que l'on donne en entrée à nos modèles, ainsi que 12 futures à prédire. A noter que nous n'avons au total que 246 séquences utilisables pour l'entrainement, et 99 pour la validation, ce qui est très peu pour entraîner un réseau neuronal.

Une première partie de notre script sert justement à générer une visualisation de ces trajectoires. Ainsi, sur les images suivantes, nous pouvons voir en bleu les 7 dernières positions observées, en rouge la dernière position observée, et en vert les 12 positions futures réelles. A noter que la frame utilisée est toujours la première de la séquence et ne correspond pas à celle utilisée pour la position en rouge, ce qui explique que les piétons n'apparaissent pas forcément sur les lignes ou sous le point rouge.





Modèles et résultats

La Baseline

La baseline est le modèle le plus simple pour ce genre de projets. Elle calcule simplement la vitesse moyenne du piéton sur ses 8 positions observées, puis prédit les 12 positions futures en supposant qu'il continue à la même vitesse et dans la même direction.

Ce modèle a généralement une ADE, la distance moyenne entre la position prédite et la position réelle, élevée car elle ne prévoit pas de changement potentiel de rythme ou de direction. Néanmoins, ici, l'ADE est de 0.3107, ce qui est très bas. Cela s'explique par le fait que les piétons marchent généralement en ligne droite, et avec un rythme constant, ce qui rend ce modèle efficace.

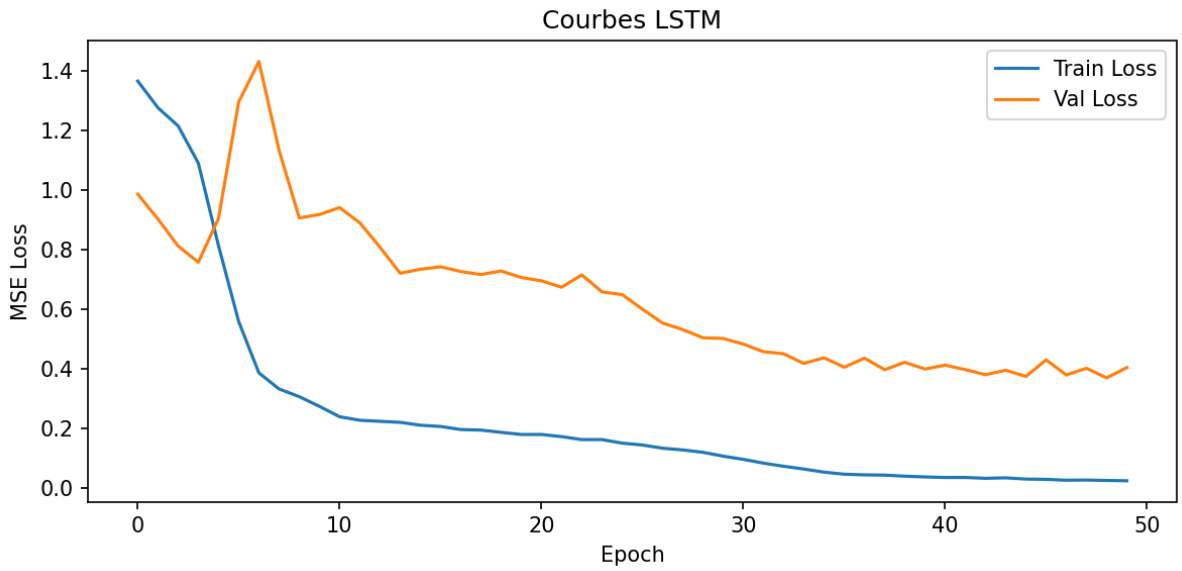
La FDE, la différence entre le dernier point prédit et le dernier point réel, est de 0.6550, ce qui est aussi logique car une légère erreur de direction au départ se creuse au fur et à mesure de l'avancement, et car la baseline est propice à ce genre de petites erreurs de départ.

Le LSTM

Le modèle suivant est le LSTM, qui agit en deux temps. Tout d'abord, l'encodeur lit les 8 positions observées une par une, retient la dynamique du mouvement (la vitesse, la direction, les accélérations) et produit un résumé. Par la suite, le décodeur utilise ce résumé pour prédire les 12 positions futures une par une.

Sur le graphique suivant, on observe qu'au fur et à mesure des époques, la courbe d'erreur sur l'entrainement chute, le modèle apprend bien sur les données qu'il connait, et la courbe d'erreur sur les séquences inconnues diminue progressivement mais reste bien supérieure à celle sur l'entrainement. Il y a donc du surapprentissage car l'échantillon d'entrainement est trop faible.

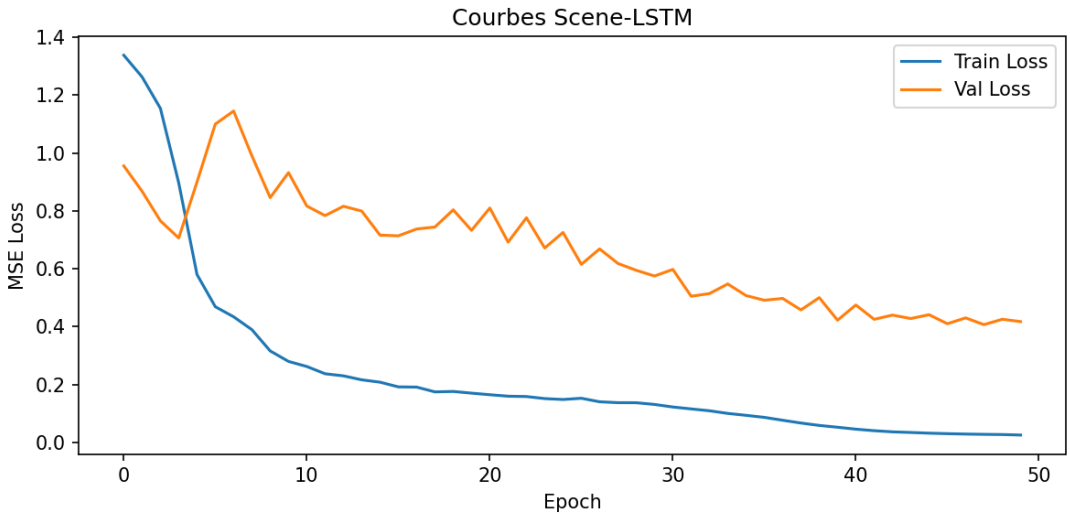
Cela explique ainsi que son ADE (0.7489) et sa FDE (1.2939) sont supérieures à celles de la baseline, les conditions sont propices à cette dernière mais pas au LSTM.



Le Scene-aware LSTM

L'avant-dernier modèle que nous utilisons est le Scene-Aware LSTM. Un défaut du LSTM classique est qu'il ne prend pas en compte les obstacles, il se base uniquement sur les coordonnées. Le Scene-Aware LSTM relève du même principe que le LSTM mais cherche à pallier ce défaut en utilisant ResNet18 pour détecter les obstacles avant de commencer la prédiction.

Néanmoins, ici nous pouvons voir sur son graphique que non seulement les résultats sont très similaires au LSTM classique (ADE = 0.7573, FDE = 1.3144), pour deux raisons. Premièrement, le Scene-Aware LSTM possède les mêmes défauts que le LSTM classique dans ce genre de cas avec peu d'entrainement, et secondement notre script donne la même image d'entrée quel que soit le piéton, le modèle ne parvient donc pas efficacement à distinguer la situation spécifique à chaque piéton.

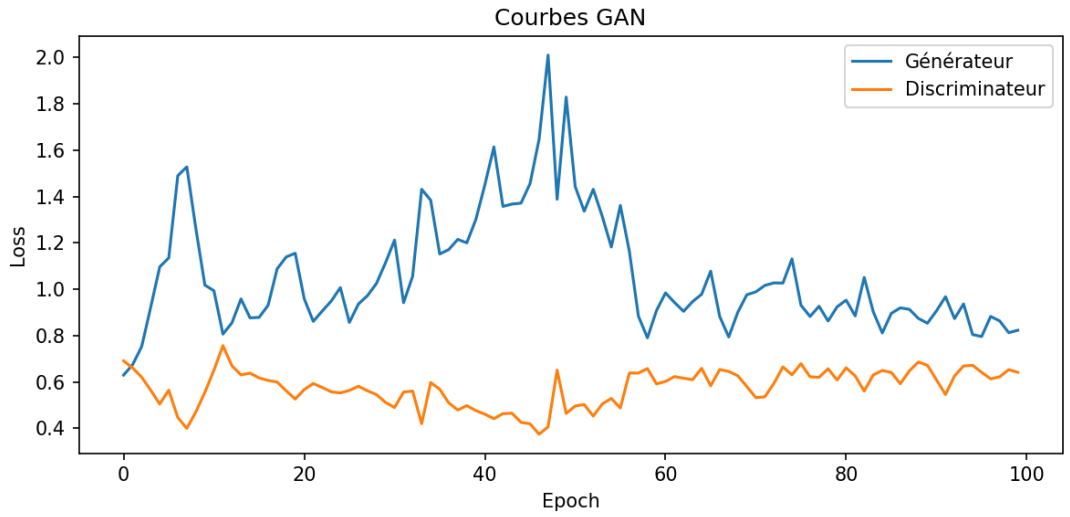


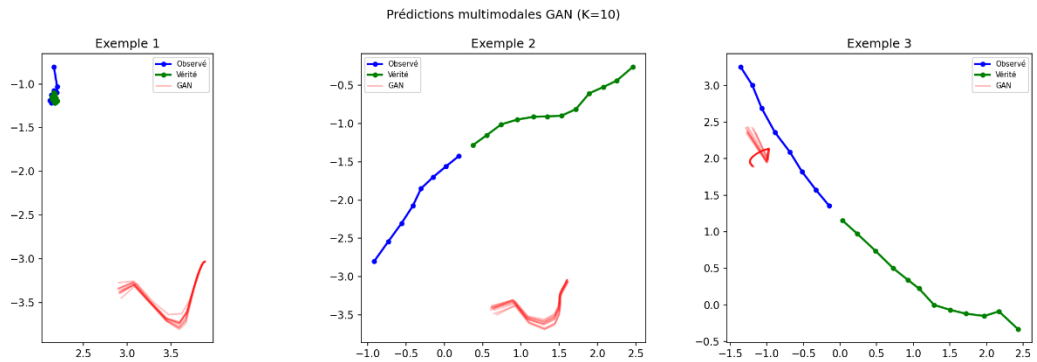
Le GAN

Le dernier modèle est le GAN, qui met en opposition le générateur et le discriminateur. Le rôle du premier est de générer des trajectoires, celui du second est de distinguer celles du dataset de celles du générateur, ce qui améliore les deux au fur et à mesure de l'entrainement.

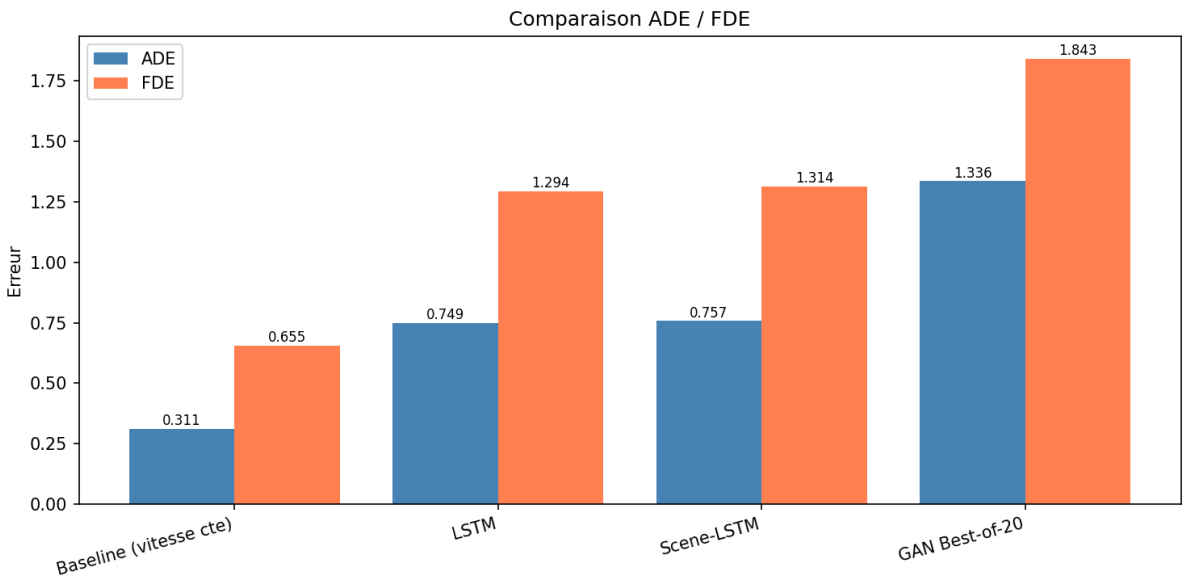
Ici, le générateur prend en entrée les 8 positions observées et un bruit aléatoire pour produire 12 positions futures. A noter qu'il prend en entrée le bruit avant de générer les positions, il ne produit pas les positions en leur appliquant chacune un bruit aléatoire. Par la suite, le discriminateur reçoit les trajectoires complètes et s'entraîne en déterminant si elle est vraie ou fausse.

Sur les courbes, on remarque que les résultats sont très instables, ce qui est normal pour ce genre d'entrainements. En utilisant la méthode Best-of-20, c'est à dire en générant 20 trajectoires par piéton et en conservant la meilleure, on obtient une ADE de 1.3363 et une FDE de 1.8434, les pires résultats du modèle. Cela est dû au fait que le générateur prend toujours un bruit similaire qui fonctionne pour tromper le discriminateur pour créer toutes ses trajectoires, et que la meilleure des 20 trajectoires n'est pas réellement différente des autres. Même si les résultats ne sont pas satisfaisants en raison d'un problème de référentiel que nous ne sommes pas parvenus à corriger, nous pouvons voir sur les prédictions suivantes que les chemins en rouges créés par le générateur sont toujours très similaires.


Courbes GAN
Générateur
Discriminateur
Loss
Epoch
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
20
40
60
80
100


Prédictions multimodales GAN (K=10)
Exemple 1
Exemple 2
Exemple 3
Observé
Vérité
GAN

Analyses


Comparaison ADE / FDE
ADE
FDE
Erreur
0.311
0.655
0.749
1.294
0.757
1.314
1.336
1.843
Baseline (vitesse cte)
LSTM
Scene-LSTM
GAN Best-of-20

Au fur et à mesure que nous testons les modèles, nous remarquons qu'aucun ne dépasse la baseline en performances, alors qu'il s'agit du modèle le plus simple.

Cela est entre autres dû à la petite taille de la base de données et du comportement des piétons. Là où le LSTM se retrouve en situation de surapprentissage avec seulement 246 séquences utilisables, la baseline se contente de continuer le mouvement tel qu'il est, ce qui est généralement l'attitude d'un piéton dans la rue.

Le Scene-Aware LSTM, quant à lui, est desservi par l'impossibilité d'utiliser efficacement à ResNet18 car tous les piétons reçoivent la même image entière sans donner d'information locale. Il obtient donc des résultats presque identiques au LSTM.

Enfin, le GAN, normalement performant, s'est fait piéger par son générateur et se trouve en mode collapse. Sa diversité est de 0.0007, quasi nulle, ce qui rend le Best-of-20 inefficace.

Difficultés rencontrées

En globalité, nous avons réussi à compléter ce projet sans trop de difficultés, mais deux d'entre elles restent à noter. Premièrement, nous avons dû effectuer le projet sur une courte période d'une semaine dont une partie pendant notre stage. Cela n'a pas en soit compliqué la tâche, mais explique notre léger retard de dépôt. Le second, plus général, a été la difficulté à créer le script, et plus précisément à trouver nos erreurs lorsqu'il y en avait. Par exemple, en essayant d'améliorer certaines parties, le temps d'exécution passait de 5 à 30 minutes (voire davantage, nous avons arrêté l'exécution à ce moment-là). Un autre exemple est la figure des trajectoires générées par le GAN, sur laquelle nous n'avons pas réussi à régler l'erreur de référentiel.

Répartition des tâches

Pour ce travail, nous avons décidé de nous répartir les différents modèles. Ainsi Timéo a débuté le projet et a intégré la baseline et le LSTM classique. Rémi s'en est ensuite servi pour intégrer le Scene-Aware LSTM et le GAN. La comparaison et l'interprétation des résultats et l'écriture du rapport se sont ensuite faites à deux après regroupement du code.

Conclusion

En conclusion, ce projet nous a permis de mieux comprendre les différents modèles d'IA que sont la baseline, le LSTM classique et Scene-Aware et le GAN, en rendant palpable des concepts qui paraissaient flous jusque-là et qui sont bien plus intéressants une fois intégrés. Cela nous a aussi permis de remarquer que selon le cas de figure, un modèle plus complexe et en apparence plus performant ne prévaut pas forcément sur un modèle plus simple. Si nous avions eu davantage de temps, nous en aurions sans doute profité pour assurer d'afficher la bonne frame sur la visualisation et ajouter les trajectoires prédites par les différents modèles.

Ainsi, grâce à ce projet, nous pourrons utiliser ces modèles d'IA en connaissant la manière dont ils fonctionnent, et avons moins d'appréhension à la perspective d'un stage dans ce domaine.